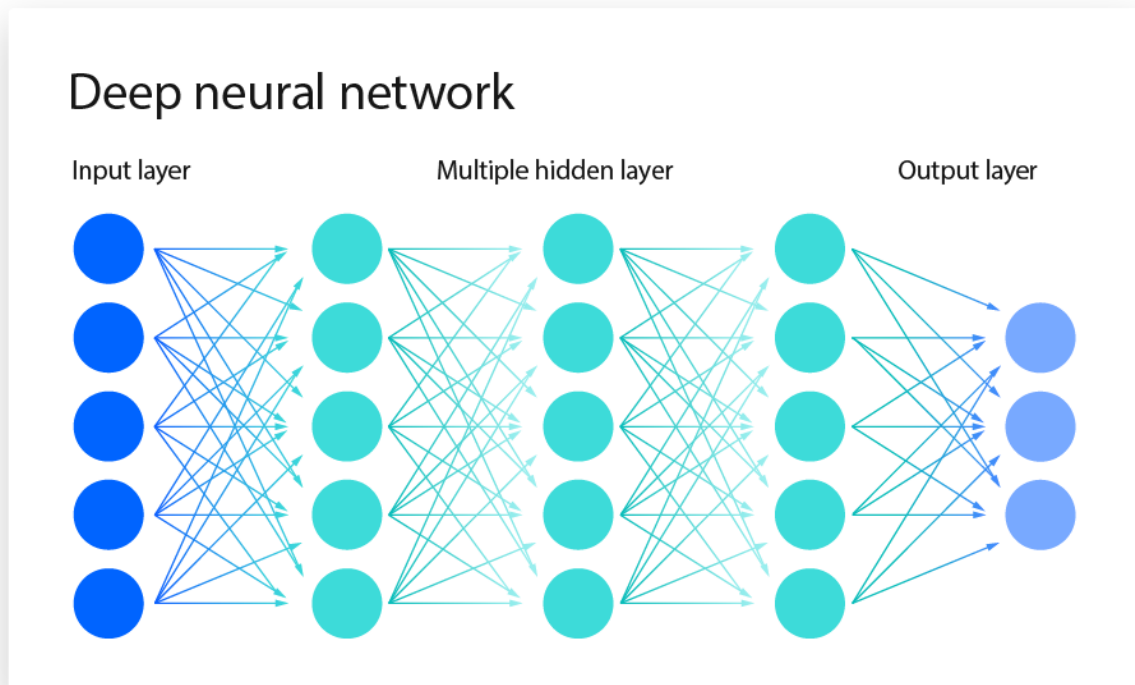


Deep Learning

Chapter 1 : Artificial Neural Networks

- Written by : Obada Mansour -

Artificial Neural Networks



1. Introduction into Deep Learning

Deep Learning is a specialized subfield of Machine Learning based on **Artificial Neural Networks (ANNs)**. While traditional algorithms require "feature engineering" (humans telling the computer what to look for), Deep Learning models are designed to automatically discover the best representations of data through multiple layers of abstraction.

1.1 From Neurons to Networks

The fundamental unit of a neural network is the **Perceptron**, which mathematically mimics a biological neuron. It receives inputs, weights them by importance, and passes them through an activation function. While a single-layer perceptron (Logistic

Regression) can only solve linearly separable problems, stacking these neurons into "hidden layers" allows the network to model highly complex, non-linear relationships.

A neural network (also called an artificial neural network) is an adaptive system that learns by using interconnected nodes or neurons in a layered structure that resembles a human brain. A neural network can learn from data—so it can be trained to recognize patterns, classify data, and forecast future events.

A neural network breaks down the input into layers of abstraction. It can be trained using many examples to recognize patterns in speech or images, for example, just as the human brain does. Its behavior is defined by the way its individual elements are connected and by the strength, or weights, of those connections. These weights are automatically adjusted during training according to a specified learning rule until the artificial neural network performs the desired task correctly.

A neural network combines several processing layers, using simple elements operating in parallel and inspired by biological nervous systems. It consists of an input layer, one or more hidden layers, and an output layer. In each layer there are several nodes, or neurons, with each layer using the output of the previous layer as its input, so neurons interconnect the different layers. Each neuron typically has weights that are adjusted during the learning process, and as the weight decreases or increases, it changes the strength of the signal of that neuron.

2. Main Neural Networks pipeline

2.1 The input layer (Data Preparation)

The pipeline begins with raw data (images, text, or numbers). Before the network can process it, the data must be converted into **tensors** (mathematical arrays).

- **Normalization:** We usually scale the data so all inputs are in a similar range (0,1).
- **Flattening:** If you have a 28 * 28-pixel image, the network "unrolls" it into a single column of 784 individual numbers.

2.2 Forward Propagation (Making a Guess)

The data flows from the input layer, through the hidden layers, to the output layer.

- Each neuron performs a calculation $z = w \cdot x + b$
- An **activation function** (like ReLU) is applied to add "logic" to the signal.
- **The Result:** The network produces a prediction, such as "90% chance this is a cat."

2.3 The Cost Function (Measuring Error)

Once the network makes its guess, it compares that guess to the "Ground Truth" (the actual label).

- The **Cost Function** calculates a single number representing the total error.
- If the error is high, the network knows its current weights and biases are poor.
- This stage is the "reality check" for the AI.

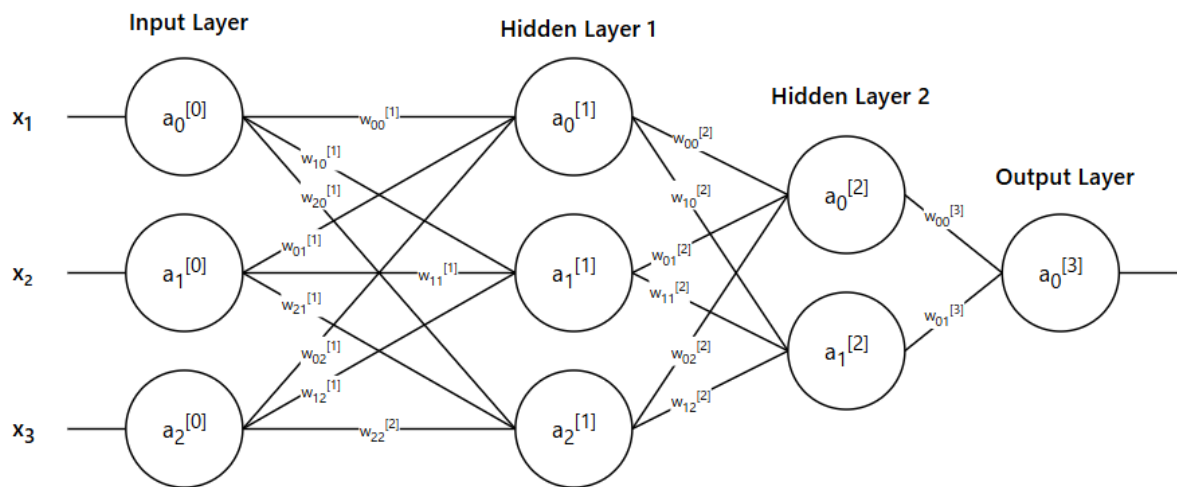
2.4 Backpropagation & Optimizer (Learning)

This is the most critical part of the pipeline. The network must now figure out **which** specific weights caused the error.

- **The Gradient:** Using calculus (specifically the Chain Rule), the network calculates the "gradient"—a direction that tells it whether to increase or decrease each weight to lower the cost.
- **The Optimizer (e.g., Adam or SGD):** This algorithm actually goes back through the layers and "tweaks" the weights by a tiny amount (the **Learning Rate**).

2.5 Repeat.

3. Neural Network Representation



3.1 The Layer Hierarchy

1. **Input Layer $a^{[0]}$** : Represents the raw features of your data (e.g., pixel values of an image or dimensions of a house). There is no computation here; it simply passes values forward note that $a^{[0]} = \vec{x}$ sample features .
2. **Hidden Layers ($a^{[1]}, a^{[2]}, a^{[3]}, \dots, a^{[L-1]}$)** : These are the internal layers that "extract" features. They are called "hidden" because their values are not observed in the training data, hidden layers are the brain of the network

3. **Output Layer $a^{[L]}$** :The final layer that produces the prediction. For binary classification, it has **1 neuron** (Sigmoid); for multi-class (like digit recognition 0–9), it has **10 neurons** (Softmax).

4. Forward Propagation Mathematics

The forward propagation through a fully connected layer l is defined by the transformation of the activation vector from the previous layer, $a^{[l-1]}$, into the current layer's activation vector, $a^{[l]}$. This process involves a linear transformation followed by a non-linear activation function σ .

4.1 Vectorized Formulation

For a layer l containing n neurons receiving input from a layer $l-1$ with m neurons (m : number of input neurons, n : number of output neurons), the operation is expressed as:

$$a^{[l]} = \sigma(z^{[l]}) = \sigma(W^{[l]} a^{[l-1]} + b^{[l]})$$

- $W^{[l]}$ is the weight matrix of shape $(n \times m)$
- $a^{[l-1]}$ is the input activation vector of shape $(m \times 1)$
- $b^{[l]}$ is the bias vector of shape $(n \times 1)$
- σ denotes the element-wise activation function (e.g., ReLU or Sigmoid).

$$a^{[l]} = \begin{bmatrix} a_0^{[l]} \\ a_1^{[l]} \\ a_2^{[l]} \\ \vdots \\ a_n^{[l]} \end{bmatrix} = \sigma \begin{bmatrix} z_0^{[l]} \\ z_1^{[l]} \\ z_2^{[l]} \\ \vdots \\ z_n^{[l]} \end{bmatrix} = \sigma \left(\begin{bmatrix} w_{00}^{[l]} & w_{01}^{[l]} & w_{02}^{[l]} & \dots & w_{0m}^{[l]} \\ w_{10}^{[l]} & w_{11}^{[l]} & w_{12}^{[l]} & \dots & w_{1m}^{[l]} \\ w_{20}^{[l]} & w_{21}^{[l]} & w_{22}^{[l]} & \dots & w_{2m}^{[l]} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ w_{n0}^{[l]} & w_{n1}^{[l]} & w_{n2}^{[l]} & \dots & w_{nm}^{[l]} \end{bmatrix} \begin{bmatrix} a_0^{[l-1]} \\ a_1^{[l-1]} \\ a_2^{[l-1]} \\ \vdots \\ a_m^{[l-1]} \end{bmatrix} + \begin{bmatrix} b_0^{[l]} \\ b_1^{[l]} \\ b_2^{[l]} \\ \vdots \\ b_n^{[l]} \end{bmatrix} \right)$$

4.2 Individual Neuron Computation :

$$a_n^{[l]} = \sigma(z_n^{[l]}) = \sigma(w_n^{T[l]} a^{[l-1]} + b_n^{[l]})$$

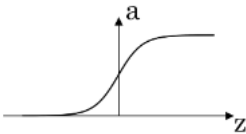
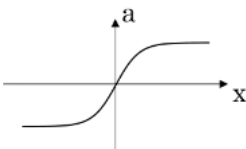
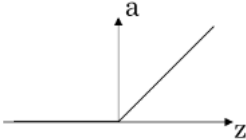
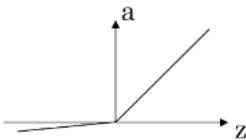
$$a_n^{[l]} = \sigma(z_n^{[l]}) = \sigma\left(\begin{bmatrix} w_{0n}^{[l]} & w_{1n}^{[l]} & w_{2n}^{[l]} & \dots & w_{mn}^{[l]} \end{bmatrix} \begin{bmatrix} a_0^{[l-1]} \\ a_1^{[l-1]} \\ a_2^{[l-1]} \\ \vdots \\ a_n^{[l-1]} \end{bmatrix} + b_n^{[l]}\right)$$

$$a_n^{[l]} = \sigma(z_n^{[l]}) = \sigma(w_{0n}^{[l]} a_0^{[l-1]} + w_{1n}^{[l]} a_1^{[l-1]} + \dots + w_{mn}^{[l]} a_m^{[l-1]} + b_n^{[l]})$$

5. Activation functions

So far, we know that a non-linear function is applied in the output step of each layer.

Actually, there are several common activation functions which are also popular.

activation	formula	graph	description
sigmoid	$a = \frac{1}{1 + e^{-z}}$		also called logistic activation function, looks like an S-shape, if your output value between 0 and 1 choose sigmoid
tanh	$a = \frac{e^z - e^{-z}}{e^z + e^{-z}}$		tanh activation usually works better than sigmoid activation function for hidden units because the mean of its output is closer to zero, and so it centers the data better for the next layer
ReLU	$a = \max(0, z)$		rectified linear unit, the most widely used activation function
Leaky ReLU	$a = \max(0.01z, z)$		an improved version of ReLU, 0.01 can be a parameter

5.1 Why ReLU (Rectified Linear Unit) is the most used activation function?

is the standard activation function in deep learning for three primary reasons:

- **Solves Vanishing Gradients**

Unlike Sigmoid or Tanh, ReLU has a constant gradient of 1 for all positive inputs. This prevents the gradient from "vanishing" (shrinking to zero) as it backpropagates through deep layers, allowing the network to keep learning.

- **Computational Efficiency (Fast)**

mathematically simple $a = \max(0, z)$ It only requires a thresholding operation at zero rather than expensive exponential calculations, making training and inference significantly faster. This makes gradient descent run much faster.

- **Encourages Sparsity**

Because any negative input results in an output of exactly zero, only a subset of neurons is active at any given time. This "sparse activation" mimics biological neurons and often leads to better model generalization

5.2 Linear function

is reference to "no activation function" .

5.3 Why do you need non-linear activation functions?

If we only allow linear activation functions in a neural network, the output will just be a linear transformation of the input, which is not enough to form a universal function approximator. If we use **no activation function at all nodes**, we are running **linear regression model**. And if we make only the output use **sigmoid** now we are running **logistic regression model**.

5.5 How to choose between Activation functions?

1. **Input layer** : doesn't contain any activation function.
2. **Hidden layer** : most used activation function in hidden layers is **ReLU** and **Sigmoid** .
3. **Output** : depending on the label y .
 - a. For binary classification : use **sigmoid function**
 - b. Regression problem : use **linear function**
 - c. Regression with $y \geq 0$: use **ReLU function**
 - d. For multi class classification : use **softmax function**

activation	formula	derivative
sigmoid	$a = \frac{1}{1 + e^{-z}}$	$a(1-a)$
tanh	$a = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1-a^2$
ReLU	$a = \max(0, z)$	0 if $z < 0$; 1 if $z \geq 0$
Leaky ReLU	$a = \max(0.01z, z)$	0.01 if $z < 0$; 1 if $z \geq 0$

6. Cost function in Neural Networks

The Cost Function (also referred to as the Objective Function or Loss Function) is a mathematical measure of the performance of the neural network. It quantifies the discrepancy between the predicted output $\hat{y} = a^{[l]}$ produced by the forward propagation and the actual target label y from the training dataset.

Loss vs. Cost

In this study, we distinguish between the **Loss function** $\mathcal{L}(\hat{y}^{(i)}, y^{(i)})$ which computes the error for a **single training example**, and the **Cost function** $J(w, b)$, which represents the average loss over the entire training set of m examples:

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

The primary goal of the training process is to find the parameters (w, b) that **minimize** $J(w, b)$.

Depending on the nature of the task (Regression vs. Classification), different functions are utilized:

A. Mean Squared Error (MSE):

For regression tasks, where the output is a continuous value, **Mean Squared Error** is standard. It squares the differences to ensure all errors are positive and to penalize outliers more heavily.

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

B. Binary Cross-Entropy (Log Loss)

For binary classification, **Cross-Entropy** is preferred. It measures the "distance" between two probability distributions. If the network is highly confident in a wrong prediction, the cost increases exponentially.

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

C. Categorical Cross-Entropy

For more than two classes classification

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^C y_j^{(i)} \log(\hat{y}_j^{(i)})$$

Where :

- m : The total number of training examples.
- $y^{(i)}$: The actual true label (Ground Truth) for the i – th example.
- $\hat{y}^{(i)}$: The predicted probability (Output) for the i – th example.
- C : The total number of classes (for multi-class tasks).

7. Backpropagation and Optimization

Backpropagation (Backward Propagation of Errors) is the central algorithm used to train neural networks. It calculates the **gradient of the loss function** with respect to each weight in the network by applying the **Chain Rule** from calculus, moving backward from the output layer to the input layer.

7.1 General Gradient Descent Algorithm

where $j = 0, 1, 2, 3, \dots, n$

Repeat until convergence {

$$\begin{aligned} w_j &:= w_j - \alpha \frac{\partial J(w, b)}{\partial w_j} \\ b &:= b - \alpha \frac{\partial J(w, b)}{\partial b} \end{aligned} \}$$

Backpropagation is calculation $\frac{\partial J(w, b)}{\partial w_j}$ and $\frac{\partial J(w, b)}{\partial b}$ using **chain rule**

Where :

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Repeat until convergence {

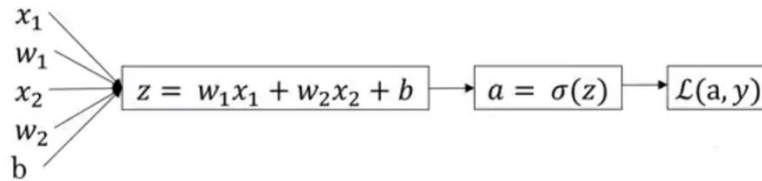
$$\begin{aligned} w_j &:= w_j - \alpha \frac{1}{m} \sum_{i=1}^m \frac{\partial \mathcal{L}(\hat{y}^{(i)}, y^{(i)})}{\partial w_j} \\ b &:= b - \alpha \frac{1}{m} \sum_{i=1}^m \frac{\partial \mathcal{L}(\hat{y}^{(i)}, y^{(i)})}{\partial b} \end{aligned} \}$$

Now we can find $\frac{\partial \mathcal{L}(\hat{y}^{(i)}, y^{(i)})}{\partial w_j}$ and $\frac{\partial \mathcal{L}(\hat{y}^{(i)}, y^{(i)})}{\partial b}$ using **chain rule**

7.2 Chain Rule in Neural Networks

$$\frac{\partial \mathcal{L}(\hat{y}^{(i)}, y^{(i)})}{\partial w_j} = \frac{\partial \mathcal{L}(\hat{y}^{(i)}, y^{(i)})}{\partial \hat{y}^{(i)}} * \frac{\partial \hat{y}^{(i)}}{\partial z} * \frac{\partial z}{\partial w_j}$$

7.2.1 Example for one neuron (logistic regression) chain rule :



Where $j = 1, 2$

$$\frac{\partial L(a, y)}{\partial w_j} = \frac{\partial L(a, y)}{\partial a} * \frac{\partial a}{\partial z} * \frac{\partial z}{\partial w_j}$$

And :

$$\frac{\partial L(a, y)}{\partial a} = \frac{a - y}{a(a - y)}, \quad \frac{\partial a}{\partial z} = a(a - y), \quad \frac{\partial z}{\partial w_j} = x_j$$

So, we get :

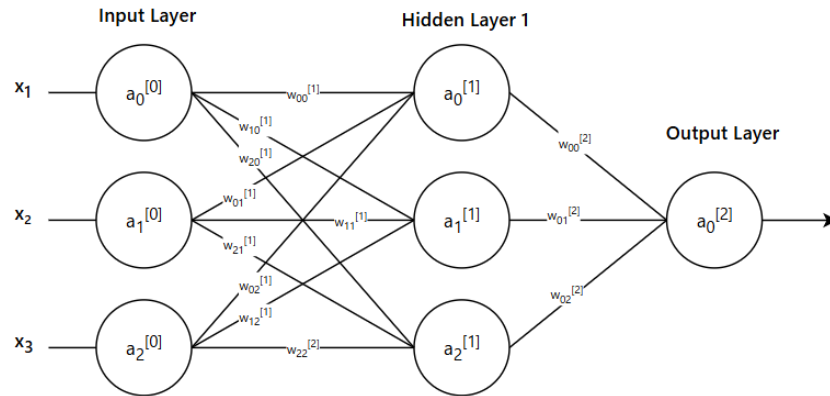
$$\frac{\partial L(a, y)}{\partial w_j} = (a - y)x_j$$

and for m training examples :

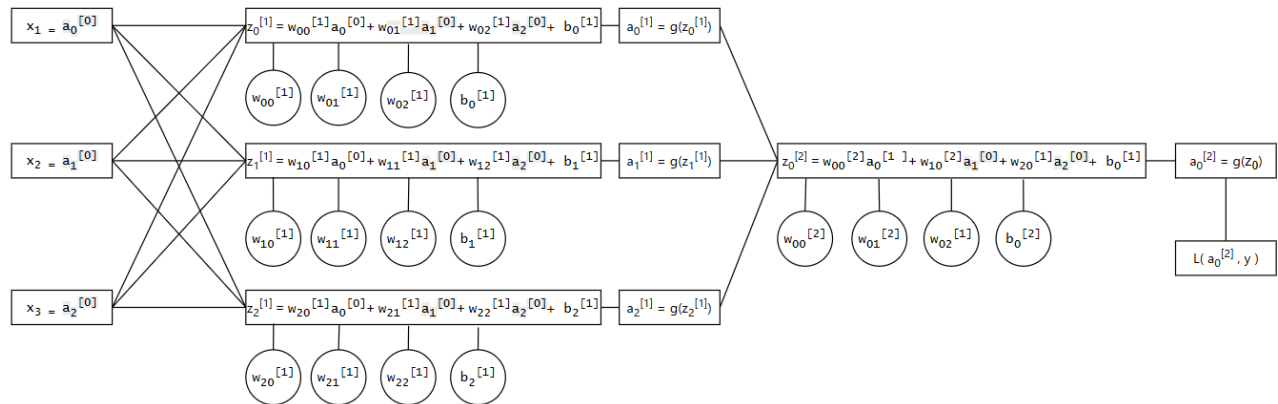
$$\frac{\partial J(w, b)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m \frac{\partial L(a^{(i)}, y^{(i)})}{\partial w_j}$$

$$\boxed{\frac{\partial J(w, b)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (a^{(i)} - y^{(i)}) x_j^{(i)}}$$

7.2.2 More complex example :



Using computation graph, the forward computation process is like this :



Let's apply backpropagation to find $w_{00}^{[2]}$ and $w_{00}^{[1]}$:

$$\frac{\partial L(a_0^{[2]}, y)}{\partial w_{00}^{[2]}} = \frac{\partial L(a_0^{[2]}, y)}{\partial a_0^{[2]}} * \frac{\partial a_0^{[2]}}{\partial z_0^{[2]}} * \frac{\partial z_0^{[2]}}{\partial w_{00}^{[2]}}$$

$$\frac{\partial L(a_0^{[2]}, y)}{\partial w_{00}^{[1]}} = \frac{\partial L(a_0^{[2]}, y)}{\partial a_0^{[2]}} * \frac{\partial a_0^{[2]}}{\partial z_0^{[2]}} * \frac{\partial z_0^{[2]}}{\partial a_0^{[1]}} * \frac{\partial a_0^{[1]}}{\partial z_0^{[1]}} * \frac{\partial z_0^{[1]}}{\partial w_{00}^{[1]}}$$

This how chain rule work in term of neural network.

7.3 Gradient Descent in Neural Networks

for make this simple we will assume that we work for 1 sample we will not use any indexing. Remember that $W^{[l]}$ is the weight matrix of shape $(n \times m)$, $b^{[l]}$ is the bias vector of shape $(n \times 1)$

Algorithm :

Repeat until convergence {

$$\begin{aligned} W^{[l]} &:= W^{[l]} - \alpha \frac{\partial J(w, b)}{\partial W^{[l]}} \\ b^{[l]} &:= b^{[l]} - \alpha \frac{\partial J(w, b)}{\partial b^{[l]}} \end{aligned} \}$$

We have the **Cost Function** $J(w, b)$ for m samples :

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(a^{[l](i)}, y^{(i)})$$

And **Loss Function** $\mathcal{L}(a^{[l](i)}, y^{(i)})$ for $i - th$ sample :

$$\mathcal{L}(a^{[l](i)}, y^{(i)}) = -[y \log(a^{[l](i)}) + (1 - y^{(i)}) \log(1 - a^{[l](i)})]$$

We will assume that we have one sample only $a^{[l](i)} = a^{[l]}$ so :

$$J(w, b) = \mathcal{L}(a^{[l]}, y) = -[y \log(a^{[l]}) + (1 - y) \log(1 - a^{[l]})]$$

Using **Chain Rule** :

$$\frac{\partial J(w, b)}{\partial W^{[l]}} = \underbrace{\frac{\partial J(w, b)}{\partial a^{[l]}} * \frac{\partial a^{[l]}}{\partial z^{[l]}}}_{\delta^{[l]}} * \underbrace{\frac{\partial z^{[l]}}{\partial W^{[l]}}}_{(a^{[l-1]})^T}$$

$$\sigma'(z^{[l]}) = \frac{\partial a^{[l]}}{\partial z^{[l]}}$$

$$\delta^{[l]} = \frac{\partial J(w, b)}{\partial a^{[l]}} * \sigma'(z^{[l]})$$

$$(a^{[l-1]})^T = \frac{\partial z^{[l]}}{\partial W^{[l]}}$$

$$\boxed{\frac{\partial J(w, b)}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^T}$$

$$\frac{\partial J(w, b)}{\partial b^{[l]}} = \frac{\partial J(w, b)}{\partial a^{[l]}} * \frac{\partial a^{[l]}}{\partial z^{[l]}} * \frac{\partial z^{[l]}}{\partial b^{[l]}}$$

$$\frac{\partial z^{[l]}}{\partial b^{[l]}} = 1$$

$$\boxed{\frac{\partial J(w, b)}{\partial b^{[l]}} = \delta^{[l]}}$$

Repeat until convergence {

$$W^{[l]} := W^{[l]} - \alpha \delta^{[l]} (a^{[l-1]})^T$$

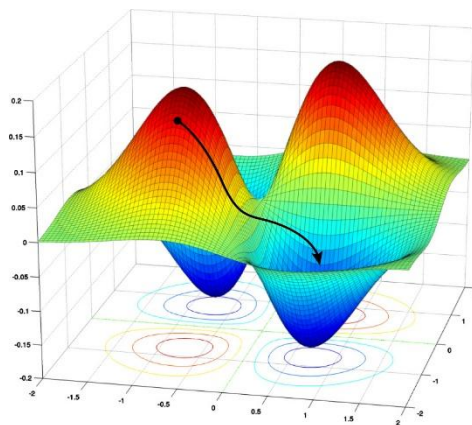
$$b^{[l]} := b^{[l]} - \alpha \delta^{[l]} \}$$

And for m samples :

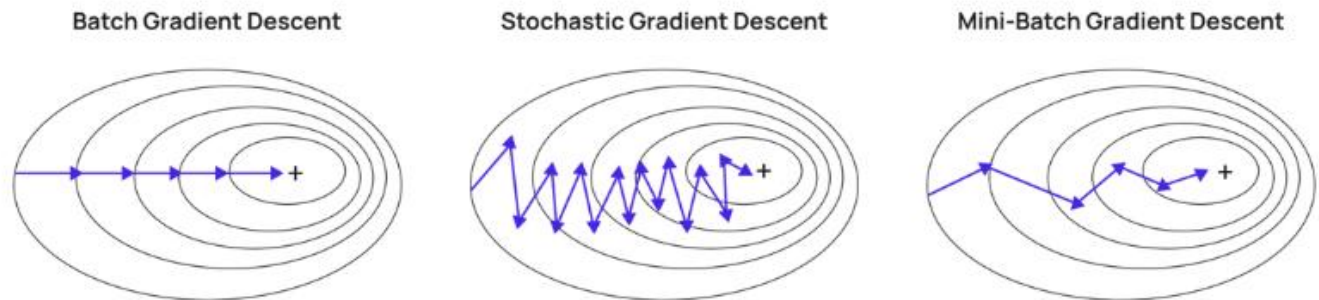
Repeat until convergence {

$$W^{[l]} := W^{[l]} - \alpha \frac{1}{m} \sum_{i=1}^m \delta^{[l](i)} (a^{[l-1](i)})^T$$

$$b^{[l]} := b^{[l]} - \alpha \frac{1}{m} \sum_{i=1}^m \delta^{[l](i)} \}$$



7.4 Main Types of Gradient Descent :



1. Batch Gradient Descent (Vanilla GD)

In this version, you calculate the gradient using the **entire dataset** (all m examples) before making a single update to the weights.

- **Process:** Go through every single training example → Calculate the average gradient → Update W and b once.
- **Pros:** Very stable; guaranteed to converge to the minimum for convex functions.
- **Cons:** Extremely slow if you have millions of images or data points. It also requires a lot of memory (RAM) to hold the whole dataset at once.

2. Stochastic Gradient Descent (SGD)

This is the opposite extreme. You update the weights immediately after looking at each individual training example.

- **Process:** Look at example 1 → Update W Look at example 2 → Update W again.
- **Pros:** Very fast and can "jump" out of local minima because the updates are noisy.
- **Cons:** The path to the minimum is very "zigzag" or "jittery." It never truly settles at the exact minimum; it just bounces around it

3. Mini-Batch Gradient Descent

This is the **standard** used in modern Deep Learning (**PyTorch**, **TensorFlow**). You split the dataset into small groups called "batches" (usually sizes like 32, 64, or 128).

- **Process:** Look at 32 examples → Calculate average gradient → Update W .Repeat for the next 32.
- **Pros:** The best of both worlds. It is faster than Batch GD and more stable than SGD. It also takes advantage of GPU "parallelization" (calculating 32 things at once).
- **Cons:** You have to choose an extra setting (the "Batch Size").

7.5 Adam Optimizer

Adam is one of the most effective optimization algorithms in deep learning. It combines:

- Momentum → smooths gradients
- RMSProp → adapts learning rate per parameter

So instead of using a single global learning rate, Adam adjusts the step size for each weight dynamically.

7.5.1 Why Adam is Powerful

1. Adaptive learning rate
 - Each parameter gets its own step size
2. Faster convergence
 - Works well even with noisy gradients
3. Less tuning needed
 - Default values usually work well